

Laporan Proyek untuk Asesmen Sertifikasi BNSP

Agentic RAG — Sistem AI Assistant Lokal

Retrieval-Augmented Generation dengan Agent Orchestrator,
berjalan sepenuhnya di infrastruktur sendiri

Penyusun tik

Instansi Pemerintah Kabupaten Hulu Sungai Selatan

Periode 21 – 23 September 2026

Repositori github.com/muhnas013/agentik-rag-project

1. Ringkasan Eksekutif

Dokumen ini melaporkan perancangan, pembangunan, dan pengujian sebuah **AI Assistant berbasis Agentik RAG** yang seluruh komponennya berjalan di mesin sendiri. Sistem menerima pertanyaan berbahasa Indonesia, lalu memilih sendiri cara menjawabnya: mencari pada dokumen yang telah diindeks, membaca teks dari gambar, atau mengambil data dari basis data relasional.

Perbedaan pokoknya dengan RAG konvensional terletak pada **Agent Orchestrator**: model bahasa tidak sekadar menerima konteks yang sudah disiapkan, melainkan memutuskan sendiri tool mana yang dipanggil untuk setiap pertanyaan.

Capaian terukur

Aspek	Hasil
Pengujian otomatis	194 test lulus , seluruhnya deterministik
Matriks uji fungsional (PRD §17)	6/7 pada keadaan biasa, 7/7 setelah model dipanaskan
Mutu retrieval bahasa Indonesia	peringkat-1 benar 6/10 → 9/10 setelah hybrid search
Perutean tool oleh Agent	4/4 pada demo terhadap sistem berjalan
Ketahanan terhadap kebocoran system prompt	0 dari 3 percobaan berhasil menembus
Endpoint REST	14
Keputusan arsitektur terdokumentasi	18 (D-01 ... D-18)
Bug tercatat dengan sebab dan perbaikannya	32 (B-01 ... B-32)

Ukuran pekerjaan

Komponen	Baris	Keterangan
Backend Python	5.742	termasuk 1.763 baris kode pengujian
Frontend React	2.110	JSX dan CSS
Dokumentasi	4.376	spesifikasi, catatan keputusan, catatan bug, panduan
Riwayat Git	35	commit, tiap perubahan disertai alasannya

❗ **Catatan mengenai angka di dokumen ini.** Seluruh angka pada laporan berasal dari pengukuran terhadap sistem yang benar-benar berjalan, bukan perkiraan. Di beberapa tempat dicantumkan pula hasil *sebelum* perbaikan, karena selisihnyalah yang menunjukkan apakah sebuah perubahan benar-benar bekerja.

2. Latar Belakang dan Tujuan

Instansi pemerintah menyimpan dokumen kebijakan, pedoman, dan arsip yang sebagian bersifat internal. Memanfaatkan layanan AI publik untuk menelusurinya berarti mengirim isi dokumen itu ke pihak ketiga. Kebutuhan yang mendasari proyek ini adalah kemampuan bertanya kepada dokumen sendiri **tanpa satu byte pun meninggalkan mesin**.

Tujuan

- 1. Membangun AI Assistant yang seluruhnya berjalan lokal.
- 2. Menyatukan tiga sumber jawaban — dokumen, gambar, dan basis data — di bawah satu antarmuka percakapan.
- 3. Membuat jawaban dapat ditelusuri: setiap kutipan menyebut berkas asalnya.
- 4. Menegakkan pembatasan akses dan keamanan pada tingkat sistem, bukan sekadar pada antarmuka.

Batasan yang ditetapkan sejak awal

- Tidak memakai layanan AI berbayar maupun API pihak ketiga.
- Berjalan pada satu laptop dengan GPU 8 GB.
- Bahasa utama antarmuka dan dokumen: bahasa Indonesia.

3. Arsitektur Sistem

Antarmuka Chat React + Vite		
↓ REST + JWT ↓		
FastAPI — main.py validasi skema · otorisasi tiga peran		
↓		
Agent Orchestrator LangChain create_agent — memilih tool sendiri		
RAG_Search hybrid: vektor + teks penuh	Image_OCR PaddleOCR di CPU	SQL_Query hanya SELECT

PostgreSQL + pgvector indeks HNSW & GIN	Folder unggahan nama diganti UUID	PostgreSQL user read-only
Ollama — LLM dan embedding, berjalan di host demi akses GPU		

Agent menerima pertanyaan beserta daftar berkas yang tersedia, lalu memilih tool. Hasil tool dikembalikan kepada Agent sebagai **data** — bukan perintah — sebelum jawaban akhir disusun.

4. Teknologi yang Digunakan

Lapis	Teknologi	Alasan pemilihan
API	FastAPI, Pydantic	validasi skema otomatis dan dokumentasi OpenAPI
Agent	LangChain create_agent	tool calling baku, satu jalur pemanggilan LLM
LLM	Ollama — qwen2.5:3b-instruct-q4_K_M	tool calling andal dan bahasa Indonesia memadai pada VRAM 8 GB
Embedding	nomic-embed-text (768 dimensi)	ringan, berjalan berdampingan dengan LLM
Basis data	PostgreSQL 16 + pgvector	vektor dan data relasional dalam satu mesin basis data
OCR	PaddleOCR (CPU)	sengaja di CPU agar tidak berebut VRAM dengan LLM
Frontend	Vite, React 19, Tailwind 4	waktu muat cepat, tanpa pustaka komponen tambahan
Kemasan	Docker Compose	basis data dan backend; Ollama di host demi akses GPU

Perangkat keras pengembangan

NVIDIA GeForce RTX 4060 Laptop, VRAM 8.188 MiB. Pengukuran langsung menunjukkan model LLM menempati 2,16 GB dan pemuatan pertama memerlukan 2,6 detik; permintaan berikutnya dilayani dalam 0,04 detik selama model masih berada di memori.

5. Rancangan Basis Data

Potongan dokumen, vektor embedding, dan indeks pencarian teks penuh berada pada satu tabel. Menyatukannya berarti tidak ada sinkronisasi antar-penyimpanan yang bisa tertinggal.

Tabel documents		
id	bigint	primary key
filename	varchar(255)	nama asli berkas
content	text	isi satu potongan
embedding	vector(768)	lebar mengikuti EMBEDDING_DIM, tidak ditulis mati
metadata	jsonb	nomor potongan, model embedding, path sumber
content_tsv	tsvector	GENERATED ALWAYS AS (to_tsvector('indonesian', content))
created_at	timestamp	
Indeks		
ix_documents_embedding_hnsw	hnsw (embedding vector_cosine_ops) m=16 ef=64	
ix_documents_content_tsv	gin (content_tsv)	

Kolom content_tsv dihitung PostgreSQL sendiri, bukan diisi kode aplikasi. Dengan begitu tidak ada jalur kode yang bisa lupa memperbaruinya ketika isi dokumen berubah.

Indeks **HNSW** dipilih, bukan IVFFlat, karena tidak memerlukan pembangunan ulang saat jumlah baris bertambah banyak.

6. Implementasi Fitur Utama

6.1 Pipeline RAG dan hybrid search

Dokumen yang diunggah diekstraksi teksnya, dipotong (800 karakter, tumpang tindih 120), diubah menjadi vektor, lalu disimpan. Pencarian menggabungkan **dua jalur**: kemiripan vektor dan pencarian teks penuh PostgreSQL, disatukan dengan *Reciprocal Rank Fusion*.

Konfigurasi	Peringkat-1 benar	Benar dalam 3 teratas
-------------	-------------------	-----------------------

Vektor saja	6/10	8/10
Teks penuh saja	9/10	10/10
Hybrid, bobot 1 : 2	9/10	10/10

Bobot kedua jalur sengaja tidak sama. Penggabungan berbobot sama justru menghasilkan 8/10 — lebih buruk daripada teks penuh sendiri, karena jalur yang lebih lemah menarik hasil yang benar ke bawah. Angka 1 : 2 berasal dari pengukuran, bukan tebakan.

Jalur vektor tetap dipertahankan meski sendiri kalah, sebab keduanya gagal pada hal yang berbeda: pencarian teks penuh buta terhadap pertanyaan yang tidak berbagi satu kata pun dengan dokumennya.

6.2 Agent Orchestrator dan ketiga tool

Tool	Fungsi	Pengamanan
RAG_Search	mencari pada dokumen terindeks	potongan bermuatan prompt injection dikarantina
Image_OCR	membaca teks dari gambar	path dibatasi ke folder unggahan
SQL_Query	SELECT ke tabel yang diizinkan	user read-only, allowlist tabel, LIMIT paksa, timeout

Agent memilih tool sendiri. Jawaban menyertakan tool yang benar-benar dipanggil beserta potongan dokumen sumbernya, sehingga setiap jawaban dapat ditelusuri kembali ke asalnya.

6.3 Antarmuka pengguna

Antarmuka percakapan menampilkan rencana tool, jawaban ter-render Markdown, dan panel sumber berisi nama berkas beserta skor kemiripannya. Skor digambarkan sebagai bilah relatif terhadap potongan terbaik — angka 0,042 tidak berarti apa pun bagi pembaca sampai ia tahu rentangnya.

Berkas yang diunggah menempel sebagai kartu di atas kolom pertanyaan, dan pengguna bebas bertanya apa saja di bawahnya. Riwayat percakapan tersimpan dan dapat dibuka kembali.

7. Keamanan

Keamanan dirancang berlapis dan ditegakkan pada tingkat sistem, bukan hanya pada antarmuka. Seluruh lapisan berikut diverifikasi terhadap sistem yang berjalan.

Risiko	Penanganan	Hasil verifikasi
--------	------------	------------------

Perubahan data lewat SQL Tool	user PostgreSQL read-only terpisah, allowlist tabel, validasi kata kunci	DELETE FROM pegawai ditolak validasi dan oleh PostgreSQL; 12 baris tetap 12
Prompt injection dari isi dokumen	potongan yang memuat pola pengambilalihan peran dikarantina sebelum masuk konteks	hasil tool diperlakukan sebagai data, bukan perintah
Kebocoran system prompt	larangan eksplisit, ditambah pendeteksi kutipan berbasis <i>shingle</i> 8 kata	0 dari 3 percobaan berhasil, termasuk yang mengaku sebagai pengembang
Berkas unggahan berbahaya	allowlist ekstensi, pemeriksaan <i>magic number</i> , batas ukuran, nama diganti UUID	berkas .exe ditolak sebelum menyentuh disk
Akses tanpa wewenang	JWT dengan tiga peran: ADMIN, USER, READ_ONLY	READ_ONLY: bertanya 200, menambah dokumen 403 , mengelola akun 403
Kebocoran data ke luar	seluruh model berjalan lokal; jalur ke API pihak ketiga dihapus (D-17)	tidak ada kredensial API pada konfigurasi maupun kode

Keputusan D-17 layak disebut tersendiri. Sistem sempat memiliki lapisan provider yang dapat dialihkan ke API luar. Lapisan itu **dihapus**, bukan sekadar dinonaktifkan: selama ia ada, isi dokumen dapat dikirim keluar hanya dengan mengubah satu baris berkas konfigurasi, tanpa jejak apa pun di kode.

8. Pengujian

8.1 Strategi dua lapis

Pengujian dipisah menurut sifat yang diuji, karena keduanya memerlukan perlakuan berbeda.

Lapis	Jumlah	Sifat	Yang diperiksa
Test otomatis (pytest)	194	deterministik, memakai tiruan	kontrak HTTP, validasi, otorisasi, logika pemotongan, penggabungan hybrid, alur keputusan Agent

Matriks uji PRD §17	7 kasus	terhadap sistem hidup, diulang	perutean tool, ketepatan jawaban, penolakan serangan
---------------------	---------	--------------------------------	--

Pemisahan ini disengaja. Jawaban model tidak deterministik, sehingga menguji mutu jawaban dengan pytest akan menghasilkan test yang kadang lulus kadang gagal tanpa ada yang berubah. Sebaliknya, kontrak HTTP dan otorisasi harus lulus setiap kali — dan itu tidak boleh bergantung pada model.

8.2 Hasil matriks uji

Kode	Yang diuji	Hasil
RAG-001	jawaban dari dokumen terindeks	lulus
OCR-001	pembacaan teks dari gambar	lulus
SQL-001	penyusunan query oleh model	tidak stabil — lihat §10
AGENT-001	pemilihan tool yang tepat	lulus
AGENT-002	pertanyaan tanpa tool	lulus
SEC-001	penolakan kebocoran system prompt	lulus
SEC-002	penolakan perintah SQL yang mengubah data	lulus

8.3 Contoh pengukuran sebelum dan sesudah

Pada beberapa perbaikan, keberhasilannya diukur dengan mengulang kasus yang sama beberapa kali sebelum dan sesudah perubahan. Dua di antaranya:

Perbaikan	Sebelum	Sesudah
B-31 — pertanyaan tentang gambar dijawab "tidak ada di dokumen"	0/5 benar	5/5 benar
B-32 — jawaban gagal ditiru dari riwayat percakapan	gagal pada riwayat berisi 3 penolakan	4/4 benar pada riwayat yang sama

Cara ini penting: pada percobaan tunggal, B-31 sempat *berhasil* secara kebetulan. Bila pengujian berhenti di situ, bug-nya akan dinyatakan tidak dapat direproduksi dan tetap ada.

9. Rekayasa Kualitas: catatan keputusan dan catatan bug

Dua catatan dipelihara sepanjang pengerjaan dan merupakan bagian dari hasil kerja, bukan pelengkap.

9.1 Catatan keputusan (18 entri)

Setiap keputusan arsitektur dicatat beserta **masalah yang dihadapi, pilihan yang diambil, dan konsekuensinya**. Beberapa keputusan kemudian direvisi keputusan berikutnya, dan revisi itu dicatat pula — sehingga alasan di balik keadaan sekarang tetap dapat ditelusuri.

Contoh	Isi
D-03	Ollama di host, bukan container, demi akses GPU langsung
D-10	ambang relevansi ditetapkan relatif, bukan angka mati, karena tiap model embedding punya rentang skor sendiri
D-14	model LLM diganti setelah pengujian; llama3.2 menulis pemanggilan tool sebagai JSON rusak
D-16	hybrid search dengan bobot dari pengukuran
D-17	penghapusan jalur ke API luar

9.2 Catatan bug (32 entri)

Tiap bug dicatat dengan **gejala, sebab, perbaikan, dan mengapa ia lolos dari pengujian yang ada**. Bagian terakhir itu yang paling sering menghasilkan perbaikan pada cara menguji, bukan hanya pada kode.

Tiga contoh yang mewakili:

- **B-28** — pertanyaan yang memuat kata "pdf" dijawab kosong. Bukan bug kode: model menghasilkan satu token lalu berhenti, dan hanya bila daftar tool ikut dikirim. Ditemukan dengan mengganti satu kata pada pertanyaan yang sama.
- **B-30** — gambar tanpa tulisan dilaporkan "gagal membaca". Sebabnya `a or b` pada `numpy.ndarray` melempar `ValueError` alih-alih menghasilkan `False`. Lolos dari 160 test karena tiruan pada test hanya memakai *list*, tidak pernah *array* — bentuk data yang bermasalah tidak pernah diuji.
- **B-32** — jawaban gagal tersimpan di riwayat lalu ditiru pada giliran berikutnya, sehingga keagalannya berputar menguatkan diri sendiri.

10. Keterbatasan yang Diketahui

Keterbatasan berikut dicatat apa adanya karena menyembunyikannya akan menyesatkan penilaian atas sistem ini.

- **Alur multi-tool dalam satu giliran belum andal.** Bila langkahnya disebut eksplisit ("cek dokumen untuk X, lalu cek database untuk Y"), berhasil 3 dari 3. Bila hanya dirangkai dengan kata "dan", gagal 0 dari 3. Ini ketergantungan pada susunan kalimat, bukan ketidakmampuan.
- **SQL-001 tidak stabil.** Penyusunan query oleh model 3B belum konsisten. Batasnya pada kapasitas model, bukan pada lapisan pencarian.
- **Model 3B dipilih karena batas VRAM.** Naik ke qwen2.5:7b (4,68 GB) cukup mengubah satu baris konfigurasi dan diperkirakan memperbaiki dua butir di atas, tetapi belum diukur.
- **Dokumen pengetahuan belum diverifikasi.** Dokumen contoh yang dimuat ke basis pengetahuan sengaja tidak memuat angka statistik; tiap tempat yang seharusnya berisi angka ditandai agar asisten menjawab "belum ada di dokumen" alih-alih mengarang.

11. Pengoperasian

Basis data dan backend berjalan sebagai container; Ollama berjalan di host agar mendapat akses GPU langsung. Satu skrip menyalakan seluruhnya dengan urutan yang benar:

```
./jalankan.sh          # basis data, backend, Ollama, antarmuka

./jalankan.sh --tanpa-ui # tanpa antarmuka
```

Urutannya tidak boleh dibalik: Ollama diikat ke alamat gateway jaringan Docker Compose, dan alamat itu baru ada setelah container dinyalakan. Skripnya menunggu alamat tersebut muncul lebih dulu.

12. Pemetaan Kompetensi

Tabel berikut memetakan bagian pekerjaan pada laporan ini terhadap area kompetensi yang lazim diujikan. **Kolom kode unit sengaja dikosongkan** untuk diisi sesuai skema sertifikasi yang ditempuh — mencantumkan kode unit yang tidak diverifikasi pada dokumen asesmen berisiko keliru dan lebih baik dihindari.

Kode unit	Area kompetensi	Bukti pada laporan ini

	Menganalisis kebutuhan dan menetapkan ruang lingkup	§2 Latar Belakang dan Tujuan; spesifikasi prd.md
	Merancang arsitektur perangkat lunak	§3 Arsitektur; 18 catatan keputusan beserta konsekuensinya
	Merancang basis data	§5 Rancangan Basis Data; pemilihan indeks HNSW dan GIN
	Membangun antarmuka pemrograman aplikasi	§6; 14 endpoint REST dengan validasi skema
	Menerapkan pemrosesan bahasa alami / model AI	§6.1 pipeline RAG; §6.2 Agent dan tool calling
	Mengolah data tidak terstruktur	ekstraksi PDF dengan tata letak, OCR gambar, pemotongan teks
	Membangun antarmuka pengguna	§6.3; pengujian di peramban pada lebar 1280px dan 420px
	Menerapkan keamanan perangkat lunak	§7; enam lapis, seluruhnya diverifikasi
	Melaksanakan pengujian perangkat lunak	§8; 194 test otomatis dan matriks uji fungsional
	Melakukan penelusuran dan perbaikan kesalahan	§9.2; 32 bug dengan sebab, perbaikan, dan pengukuran
	Mengelola konfigurasi dan penerapan	§11; Docker Compose, skrip operasional, 35 commit
	Menyusun dokumentasi teknis	4.376 baris dokumentasi: spesifikasi, keputusan, bug, panduan

13. Penutup

Sistem yang dilaporkan telah berjalan dan diuji terhadap keadaan sesungguhnya, bukan hanya pada lingkungan pengujian. Nilai utamanya bukan pada kebaruan masing-masing komponen — RAG, OCR, dan tool calling semuanya teknik yang sudah mapan — melainkan pada penyatuannya menjadi satu sistem yang berjalan lokal, dapat ditelusuri jawabannya, dan pembatasan aksesnya ditegakkan sampai ke tingkat

basis data.

Hal yang paling banyak menentukan mutu hasil selama pengerjaan adalah kebiasaan **mengukur sebelum dan sesudah** tiap perbaikan, serta mencatat alasan setiap keputusan. Beberapa perbaikan yang tampak berhasil pada satu percobaan terbukti gagal ketika diulang, dan tanpa pengulangan itu keduanya akan dinyatakan selesai padahal belum.

Laporan ini disusun berdasarkan keadaan repositori pada 23 September 2026. Seluruh angka dapat diperiksa ulang dengan menjalankan pengujian pada repositori yang sama.